
A BENCHMARK FOR AI-ACCELERATED OCEAN SPIN-UP

A PREPRINT

April 15, 2026

ABSTRACT

Spinning up an ocean general circulation model to a statistical equilibrium is among the most computationally expensive steps in climate modelling, often requiring thousands of simulated years to reach a drift-free baseline. A growing body of work uses machine learning to shortcut this procedure by predicting equilibrated ocean states, yet comparisons across methods remain difficult: each study uses its own configuration, forcing, diagnostics, and restart-file handling. We present **nemo-spinup-bench**, an open benchmark for AI-accelerated spin-up of the NEMO ocean model. The benchmark decomposes the problem into three reusable components: forecasting, restart-file construction, and physics-based evaluation, each exposed as a standalone Python package with a pluggable strategy interface, reproducible pipelines, and continuous integration. We ship reference baselines (PCA with Gaussian-process forecasting) on the idealised DINO configuration, along with a published dataset of NEMO outputs and restarts. By separating the ML forecaster from the surrounding numerical machinery, the benchmark lets new methods (including deep generative models and neural emulators) be evaluated against a common yardstick, with planned extensions to OM4 and Oceananigans.

Keywords Ocean spin-up · NEMO · Benchmark · Machine learning · Climate modelling · DINO

1 Introduction

Recent progress in AI-driven oceanography spans neural general circulation models (?), global ocean emulators (?), and deep generative approaches for producing physically consistent ocean states (??). Progress in this area is increasingly supported by open benchmarks such as OceanBench for short-range forecasting (?), and by shared idealised model configurations such as the DINO diabatic pole-to-pole setup (?) used throughout this paper.

‘Spin-up’ is the process by which a model is run forward from an initialization state to a stable state. It is a long and computationally expensive step for ocean and climate models. It takes a lot of resources to reach a stable state (of the order to 0.5M CPU-hours for a global ocean model) and has to be repeated for each new experiment (about 5 experiments/year at IPSL), which means that the centre consumes 2.5M cpu-hours/year just to bring the model from an initial state (often arbitrarily taken as having no ocean current and a climatological present-day temperature/salinity) to a stable state that we can analyse, not to mention the human labour to launch and monitor all these runs. This is a global problem at the scale of any centre in the world for oceanic and land components; both have long spin-up phases. Thus, accelerating the spin-up (even by a small amount) by generating better initial states for the ocean models could have a tremendous impact on the field.

Accelerating spin-up is both a machine learning problem (generating new states) and a computer science problem (efficiently delivering these new states to the model). The machine learning sophistication of the spin-up problem is very high; it amounts to generating new, high-dimensional states that are physically stable and closer to convergence than the usual initialization. However, with advances in generative learning and the rise of climate emulators, we can realistically hope that deep learning will bring new solutions to the table. History tells us [imagenet, davis, voc, mnist, celeb ...] that the best way to really address a rich deep learning challenge is to build a benchmark and open it for researchers to compete; this will foster a variety of approaches for the deep learning community and is likely to be far more efficient than working solely on the field with the climate science community. The spin-up task is close to image generation or long-term video forecasting, which attract many researchers, although it is rather daunting for researchers

outside of oceanography to start due to the complexity of evaluating new states which requires interacting with ocean models

2 Related work

3 The spin-up problem

4 Benchmark design

`nemo-spinup-bench` defines a reproducible, end-to-end task for evaluating machine-learning-based acceleration of ocean spin-up. The current benchmark is built around the DINO idealised diabatic configuration of NEMO (?), which offers the minimum complexity required to exhibit the large-scale circulation patterns relevant to spin-up while being small enough to permit rapid iteration. Extensions to OM4 and Oceananigans are planned as future work, and the component interfaces have been designed so that swapping the underlying ocean model amounts to providing a new variable-to-file mapping and a compatible restart writer, rather than altering the pipeline itself. A dataset containing the DINO simulation outputs, restart files, and reference evaluation outputs is published on Zenodo (?) and is pulled in as a submodule by the benchmark repository, so that any reported result is tied to a specific, citable snapshot of the data.

The benchmark deliberately fixes the task rather than the method: given a simulation window of N training time units, a method must produce a projected ocean state M time units in the future (the cadence is set by the input data: yearly, monthly, or other), from which a valid NEMO restart file is assembled and evaluated. Every component between the input window and the final restart is pluggable. The default pipeline ships a PCA + Gaussian process baseline, recovering the behaviour of the original spin-up tool (?), but any alternative dimensionality-reduction or forecasting strategy can be substituted by registering a new class and selecting it through a YAML configuration (Section ??).

4.1 Pipeline

The benchmark is composed of three interoperable tools, each released as an installable Python package with its own test suite and continuous integration:

- `nemo-spinup-forecast` performs dimensionality reduction on the training window, fits a forecaster in the reduced space, and reconstructs predictions for sea-surface height, temperature, and salinity. Each run writes to a timestamped, content-addressed output directory, so experiments accumulate without clobbering each other.
- `nemo-spinup-restart` consumes these predictions together with the mesh mask and a source restart and derives the remaining prognostic fields required by NEMO (density, velocities, surface-mean diagnostics), producing a restart file that NEMO can pick up without modification. An upsampling path based on xESMF bilinear interpolation (?) allows coarse-resolution predictions to be applied to finer target grids.
- `nemo-spinup-evaluation` scores the resulting restart against a physically-motivated reference run, computing a suite of domain metrics and optionally recomputing diagnostic quantities such as potential density from the underlying temperature and salinity fields (?).

The default example drives the full pipeline on a 50-year DINO window: baseline grids and restarts are first evaluated, a thirty-year forecast is produced from the 20 → 50 year segment, a new restart is assembled, and the final state is re-evaluated against the reference. A second example applies the same pipeline to previously published diffusion-based predictions (?), and reproduces the reported metrics to within a floating-point tolerance, providing an external check on the implementation.

4.2 Metrics

Evaluation is physics-based rather than purely statistical. For each canonical variable (temperature, salinity, sea-surface height, and derived density), the benchmark reports mean absolute error and RMSE against a reference run, computed either on grid output files, on restart files, or on both. Variable-to-file binding is driven by a YAML mapping, so the same metrics can be applied to different model configurations (for example ORCA2/1) and, with appropriate mappings, to non-NEMO outputs that adopt a similar data layout.

Two features matter for reproducibility. First, the evaluation tool validates required coordinates (including the handling of NEMO `nav_lat/nav_lon` coordinates, whose loss on variable extraction was previously observed to change a

representative temperature metric from 3.24 to 11.51). Second, large datasets are loaded lazily through Dask, with an eager-mode toggle for smaller files, making the metrics tractable on either a laptop or a high-memory node. Temporal alignment between the simulation and the reference is handled through CDO-based resampling, so simulations produced at different output frequencies can be compared on a common time axis.

5 Baselines

The forecasting framework ships two registries of interchangeable components (one for dimensionality reduction and one for forecasting) which together define the baselines shipped with the benchmark. Both registries are exposed as `name → class` dictionaries and selected at runtime through `techniques_config.yaml`.

Dimensionality reduction. Two implementations are provided out of the box:

- **PCA** (`DimensionalityReductionPCA`): linear principal-component analysis applied to the masked, standardised ocean field. The number of retained components is either a fixed integer or a target fraction of explained variance, controlled by the `-comp` CLI argument.
- **Kernel PCA** (`DimensionalityReductionKernelPCA`): a non-linear variant using scikit-learn’s kernel PCA, intended as a drop-in reference for non-linear reductions.

Forecasting. Two Gaussian-process variants are registered, both built on a shared kernel following the original formulation by Tissot and co-authors. The two wrappers differ only in how this regressor is invoked:

- **GaussianProcessForecaster**: a direct forecaster that fits the GP once on the full training window and predicts all future steps in a single call. Returns both point predictions and posterior standard deviations.
- **GaussianProcessRecursiveForecaster**: a recursive forecaster built on `skforecast’s ForecasterRecursive`, which feeds lagged targets and rolling window features back into the regressor to produce multi-step forecasts one step at a time. Uncertainty estimates are not available in this mode.

Data. The baselines are driven by a published DINO dataset hosted on Zenodo (?). Two archives are used: `200.zip` contains 200 years of annual-mean DINO output on the native grid (sea surface height, temperature, salinity, and companion fields on the T, U, V grids, together with mesh masks), and `restart.zip` contains the matching 200 annual restart files needed to seed a continuation run of NEMO. Forecasting is performed on a contiguous training window from `200.zip`, and the predicted final state is combined with the corresponding restart from `restart.zip` to assemble a physically valid restart that NEMO can read directly. A smaller `50.zip` (the first 50 years) is provided for quick smoke tests and continuous-integration runs.

Default pipeline. The packaged `techniques_config.yaml` selects **PCA** as the dimensionality-reduction technique and **GaussianProcessForecaster** as the forecaster, recovering the method of Tissot et al. These are the baselines used to produce the headline results in Section ???. Alternative combinations (PCA with the recursive GP, Kernel PCA with either forecaster) can be selected by editing a single YAML entry; adding a new baseline amounts to subclassing the appropriate base class and registering it, without changes to the CLI or the pipeline.

6 Results

7 Discussion

8 Conclusion

Data availability

The DINO dataset used throughout this paper is archived on Zenodo at <https://doi.org/10.5281/zenodo.19557419> (?). It contains the NEMO/DINO simulation outputs and restart files needed to reproduce the benchmark, together with the reference outputs used for evaluation.

References

- Dmitrii Kochkov, Janni Yuval, Ian Langmore, Peter Norgaard, Jamie Smith, Griffin Mooers, Milan Klöwer, James Lottes, Stephan Rasp, Peter Düben, Sam Hatfield, Peter Battaglia, Alvaro Sanchez-Gonzalez, Matthew Willson, Michael P. Brenner, and Stephan Hoyer. Neural general circulation models for weather and climate. *Nature*, 632 (8027):1060–1066, August 2024. ISSN 1476-4687. doi:10.1038/s41586-024-07744-y. URL <https://www.nature.com/articles/s41586-024-07744-y>.
- Surya Dheeshjith, Adam Subel, Alistair Adcroft, Julius Busecke, Carlos Fernandez-Granda, Shubham Gupta, and Laure Zanna. Samudra: An AI Global Ocean Emulator for Climate. *Geophysical Research Letters*, 52(10):e2024GL114318, May 2025. ISSN 0094-8276, 1944-8007. doi:10.1029/2024GL114318. URL <http://arxiv.org/abs/2412.03795>. arXiv:2412.03795 [physics].
- Etienne Meunier, David Kamm, Guillaume Gachon, Redouane Lguensat, and Julie Deshayes. Learning to generate physical ocean states: Towards hybrid climate modeling, February 2025. URL <http://arxiv.org/abs/2502.02499>. arXiv:2502.02499 [cs].
- Blandine Gorce, Luther Ollier, David Kamm, and Etienne Meunier. Physically Consistent Sampling For Ocean Model Initialization. In *NeurIPS 2025 Workshop on Tackling Climate Change with Machine Learning*, San Diego, CA, December 2025. URL <https://neurips.cc/virtual/2025/loc/san-diego/126936>.
- Bytez.com, Anass El Aouni, Quentin Gaudel, Juan Emmanuel Johnson, REGNIER Charly, Julien Le Sommer, van Gennip, Ronan Fablet, Marie Drevillon, Yann DRILLET, and Pierre Yves Le Traon. OceanBench: A Benchmark for Data-Driven Global Ocean For..., December 2025. URL <https://bytez.com/docs/neurips/121394/paper>.
- D. Kamm, J. Deshayes, and G. Madec. DINO: A Diabatic Model of Pole-to-Pole Ocean Dynamics to Assess Subgrid Parameterizations across Horizontal Scales. *EGUsphere*, 2025:1–26, 2025. doi:10.5194/egusphere-2025-1100. URL <https://egusphere.copernicus.org/preprints/2025/egusphere-2025-1100/>.
- Matt Archer. DINO dataset for nemo-spinup-bench, April 2026. URL <https://doi.org/10.5281/zenodo.19557419>. Ocean model outputs and restart files from NEMO simulations for training machine learning models to forecast spinup dynamics.
- Matt Archer, Isaac Akanho, Surbhi Goel, Simon Sadler, Etienne Meunier, Guillaume Gachon, and Julie Deshayes. Spinup-Evaluation: A Physics-Based Benchmarking Platform for ML-Based Spinup of the NEMO Ocean Model. In *Presented at the Ocean Sciences Meeting 2026 (OSM26)*, Glasgow, Scotland, February 2026. URL <https://agu.confex.com/agu/osm26/meetingapp.cgi/Paper/2036003>.

A Software architecture

The benchmark is implemented as four loosely-coupled repositories (`nemo-spinup-bench`, `nemo-spinup-forecast`, `nemo-spinup-restart`, and `nemo-spinup-evaluation`), each released as an installable Python package with a `src/` layout, a `pyproject.toml`, and a console entry point. The separation reflects the fact that the same forecasting method might be paired with different restart-assembly strategies when targeting different ocean models, and that the evaluation tooling is independently useful for comparing any pair of ocean simulations. A Zenodo-backed dataset of DINO outputs and reference restarts (?) is consumed by all four repositories as a submodule, so that the data, code, and metrics reported in this paper are reproducible from a single commit graph.

Pluggable strategies. Dimensionality reduction and forecasting are expressed as abstract base classes (`DimensionalityReduction` and `BaseForecaster`) with small interfaces. Concrete implementations (PCA, kernel PCA, direct and recursive Gaussian processes) are registered in a dictionary and selected by name through `techniques_config.yaml`. Adding a new strategy, including deep-learning forecasters built on PyTorch or JAX, requires only subclassing the relevant base and registering the class; no changes to the orchestration code or the CLI are needed. Variable-to-file binding is likewise externalised to `ocean_terms.yaml`, so different model configurations can be targeted without code changes.

Reproducible runs. The forecasting CLI writes each run to a timestamped, content-hashed directory under the user-supplied output path, with a `latest` symlink pointing at the most recent run. This keeps experimental artefacts isolated and makes bisection over code or configuration changes straightforward. Pipeline orchestration is driven by a list of `TermDef` specs: small immutable dataclasses that bundle an internal key, a NetCDF variable name, and a source filename, so new variables can be added to the pipeline by appending a single entry.

Continuous integration. Each repository enforces formatting and linting with `ruff`, runs `pre-commit` hooks on every push, and executes its full `pytest` suite on every pull request. Test fixtures that require real NetCDF inputs are fetched from Zenodo. The evaluation package additionally publishes versioned API documentation to Read the Docs on successful CI.

Test coverage. The combined test suites exercise PCA round-trip identity to $\approx 10^{-5}$, the NEMO restart-variable derivation, coordinate handling on real DINO NetCDFs, and the end-to-end pipeline from raw outputs to final metrics. Fixtures parametrise tests over every registered strategy, so introducing a new dimensionality-reduction or forecasting technique automatically expands the coverage matrix.